Labo 08 — Configuration et secrets

Théorie — comment une seule image sert en développement, en recette et en production sans être reconstruite, et pourquoi un mot de passe qui entre dans une image, une ligne de commande ou un log n'en ressort jamais.

Objectifs

- Comprendre pourquoi la configuration doit être **hors de l'image**, et ce que dit le principe *12-factor*.
- Maîtriser `ENV`, `-e`, `--env-file`, leur ordre de priorité et la convention `_FILE`.
- Relier variables d'environnement et configuration Spring Boot (*relaxed binding*, profils).
- Savoir **où un secret se voit** (image, `inspect`, `exec`, logs, `ps`) et donc où il ne doit pas aller.
- Utiliser `podman secret` à l'exécution et `--mount=type=secret` au build.

---

1. Une image, trois environnements

Au labo 05, vous avez construit `api:1.0`. Cette image doit tourner sur votre poste contre une base locale, sur le serveur de recette contre une base de test, et en production contre la vraie base — avec des URL, des mots de passe et des niveaux de log différents. Si chaque environnement avait sa propre image, on testerait en recette une image **qui n'est pas** celle qui part en production. Toute la discipline du déploiement repose sur l'inverse : **une image immuable, promue d'un environnement à l'autre, et une configuration injectée de l'extérieur**.

> **→ HISTOIRE**
> En 2011, les ingénieurs de Heroku publient *The Twelve-Factor App*, douze règles pour des applications déployables en nuage. Le facteur III, *Config*, est le plus cité : « stocker la configuration dans l'environnement », parce que les variables d'environnement sont indépendantes du langage, du système et du fichier de configuration, et qu'elles ne risquent pas d'être commises dans Git par accident. Docker (2013) en a fait sa pratique par défaut : `-e`.

Ce qui change entre environnements est de la **configuration** : URL de base, identifiants, ports, niveaux de log, drapeaux de fonctionnalités. Ce qui ne change pas est du **code**, et vit dans l'image. Et parmi la configuration, ce qui donne accès à quelque chose est un **secret** : mot de passe, clé d'API, certificat privé. Les trois ne se traitent pas de la même façon.

2. Les mécanismes

Mécanisme	Où	Portée	Usage
`ENV` dans le Dockerfile	dans l'image	tous les conteneurs de l'image	**valeur par défaut** raisonnable, jamais un secret
`-e CLE=valeur`	`podman run`	ce conteneur	surcharger une ou deux valeurs
`--env-file api.env`	`podman run`	ce conteneur	un jeu de valeurs par environnement, fichier hors Git
fichier monté (`-v ./application.yaml:/config/…:ro`)	`podman run`	ce conteneur	configuration structurée, longue, versionnée
`--secret` (`/run/secrets/…`)	`podman run`	ce conteneur	mots de passe, clés, certificats
argument de commande	`podman run … --mot-de-passe=x`	ce conteneur	**jamais** pour un secret (visible dans `ps`, `logs`, `inspect`)

L'ordre de priorité est fixe : **`-e` écrase `--env-file`, qui écrase `ENV` de l'image**, quel que soit l'ordre des options sur la ligne. Et `-e CLE` sans valeur transmet la variable **du shell de l'hôte** : pratique en CI, piège en relecture.

→ LINUX
Les variables d'environnement sont un bloc de texte `CLE=valeur` que chaque processus reçoit de son parent à sa création, et que tout enfant hérite. Elles ne sont ni chiffrées ni protégées : `/proc/<pid>/environ` les expose à tout processus du même utilisateur, et `podman exec conteneur env` les affiche. Un programme ne peut pas les « oublier » une fois lancé — ce qui est la raison pour laquelle un secret en variable est **lisible**, mais aussi pourquoi c'est si simple.

→ SPRING BOOT
Spring Boot lit **toutes** ses propriétés dans l'environnement grâce au *relaxed binding* : `spring.datasource.url` devient `SPRING_DATASOURCE_URL`, `server.port` devient `SERVER_PORT`, et `SPRING_PROFILES_ACTIVE=prod` active le profil `prod` (donc `application-prod.yaml`). Un fichier monté dans `/config/application.yaml` est lu automatiquement, et `SPRING_CONFIG_ADDITIONAL_LOCATION` pointe vers d'autres. L'ordre : variable d'environnement > fichier externe > fichier dans le JAR. Exactement la même logique que celle du moteur de conteneurs, un étage plus haut.

→ JAVA
La JVM lit elle aussi une variable : `JAVA_TOOL_OPTIONS`. Son contenu est ajouté aux options de démarrage, et la JVM le signale par `Picked up JAVA_TOOL_OPTIONS: …` sur la sortie d'erreur. C'est ainsi qu'on règle la mémoire (`-Xmx`, `-XX:MaxRAMPercentage`) ou un agent sans toucher à l'`ENTRYPOINT` de l'image — le labo 10 s'en sert pour les limites mémoire.

3. Où un secret se voit

Faites l'inventaire avant de choisir un mécanisme. Un secret placé…

- **dans l'image** (`ENV`, `COPY .env`, `ARG` — labo 04) est lisible par `podman history`, `podman image inspect`, `podman run --rm image cat /app/.env`, et par toute personne qui tire l'image du

registry. **Jamais.**

- **en argument de commande** (`--db-password=x`) apparaît dans `podman inspect` (`Config.Cmd`), `podman top`, le `ps` de l'hôte, et souvent dans les logs de l'application qui affiche ses arguments. **Jamais.**
- **en variable d'environnement** (`-e`, `--env-file`) apparaît dans `podman inspect` (`Config.Env`), `podman exec env`, `/proc/1/environ`, et dans tout rapport d'erreur ou *dump* qui inclut l'environnement. **Acceptable** sur un poste ou en CI, **insuffisant** en production.
- **dans un fichier monté** en lecture seule, ou un secret du moteur, n'apparaît ni dans `inspect`, ni dans l'environnement ; seul le processus qui lit le fichier le voit. C'est la cible.

PIÈGE
Les images officielles proposent presque toutes la convention `_FILE` : `POSTGRES_PASSWORD_FILE=/run/secrets/db_password` fait lire le mot de passe dans un fichier au lieu de `POSTGRES_PASSWORD`. L'environnement ne contient alors qu'un **chemin**. Vérifiez que votre propre application sait faire pareil : Spring Boot le fait avec `spring.config.import=optional:configtree:/run/secrets/`, qui transforme chaque fichier du dossier en propriété.

4. Les secrets du moteur

`podman secret create db_password -` lit un secret sur l'entrée standard et le range dans le stockage de votre utilisateur ; `podman secret ls` le liste, `podman secret inspect` en montre les métadonnées sans la valeur, `podman secret rm` le supprime. Au `run`, `--secret db_password` le monte dans le conteneur sous `/run/secrets/db_password`, en lecture seule, et **rien n'apparaît dans `podman inspect`**. La variante `--secret db_password,type=env,target=POSTGRES_PASSWORD` l'injecte en variable d'environnement pour les applications qui ne lisent pas de fichier — visible par `exec env`, mais toujours absent de `inspect`.

PODMAN
`podman secret` fonctionne sans aucun orchestrateur. Chez Docker, `docker secret` exige le mode Swarm ; hors Swarm, on se rabat sur les `secrets:` de Compose, qui sont de simples fichiers montés — et que `podman compose` comprend aussi. Côté stockage, le pilote par défaut de Podman (`file`) garde la valeur **encodée, pas chiffrée**, dans `~/.local/share/containers/storage/secrets/`, et `podman secret inspect --showsecret` la relit : c'est un rangement, pas un coffre. Quadlet (labo 10) sait y référencer un secret, et Kubernetes a son objet `Secret` (labo 14), qui a la même limite.

→ SÉCURITÉ
En entreprise, les secrets vivent dans un **gestionnaire de secrets** (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) : chiffrés au repos, accès journalisé, **rotation** automatique. Le conteneur reçoit soit un fichier injecté au démarrage, soit un jeton de courte durée pour aller chercher le secret lui-même. L'objectif n'est pas « que personne ne puisse le lire », c'est « savoir qui l'a lu, et pouvoir le changer en une minute ».

5. Les secrets de build

Un `npm install` ou un `mvn package` a parfois besoin d'un jeton pour un dépôt privé, **pendant** le build seulement. Le labo 04 a montré que `ARG` finit dans `podman history`. La bonne réponse est le montage de secret : `RUN --mount=type=secret,id=token,required=true cat /run/secrets/token` rend le fichier disponible pour **cette seule instruction**, et `podman build --secret id=token,src=token.txt` le fournit. Il n'est écrit dans aucune couche, `history` ne montre que la

ligne `--mount=…`, et `/run/secrets` n'existe plus dans l'image. Sans `--secret`, `required=true` fait échouer le build : `secret required but no secret with id "token" found`.

Attention au cache : une instruction `RUN` déjà en cache **n'est pas réexécutée**, donc ne redemande pas le secret. Un jeton expiré ne se remarque qu'au prochain `--no-cache`.

6. Le fichier `.env`

Un fichier `api.env` (ou `.env`) par environnement est la forme la plus répandue : une ligne `CLE=valeur` par variable, `#` pour les commentaires, pas de guillemets ni d'espaces autour du `=`. Trois règles : il est dans `.gitignore` **et** dans `.dockerignore` (labo 04) ; un `api.env.example` sans valeurs est commis à sa place ; et il ne contient pas de secrets de production — pour ceux-là, un secret ou un fichier monté.

7. En entreprise

- **Une image, promue** : la CI construit `api:1.4.2` une fois, la pousse, et le même digest est déployé en recette puis en production avec des variables différentes.
- **Les profils Spring par variable** : `SPRING_PROFILES_ACTIVE=prod` dans le déploiement, jamais dans le Dockerfile ; `application-prod.yaml` ne contient pas de mot de passe mais `${DB_PASSWORD}`, résolu depuis l'environnement ou un fichier de secrets.
- **Les secrets** : fichiers injectés par l'orchestrateur ou le gestionnaire de secrets, `_FILE` ou `configtree`, rotation planifiée. Un secret qui apparaît dans un log, un ticket ou un `inspect` est considéré **compromis** et changé.
- **La revue** : un `ENV *_PASSWORD` ou un `COPY .env` dans un Dockerfile est refusé en revue de code, automatiquement si possible (*linters* Hadolint, scanners Trivy/Gitleaks).

À retenir

- Une image immuable, une configuration injectée : `ENV` pour les défauts, `-e` / `--env-file` pour l'environnement, fichiers ou secrets pour ce qui est sensible.
- Priorité fixe : `-e` > `--env-file` > `ENV` de l'image.
- Spring Boot lit tout dans l'environnement (`SPRING_DATASOURCE_URL`, `SPRING_PROFILES_ACTIVE`) ; la JVM lit `JAVA_TOOL_OPTIONS`.
- Un secret en argument se voit dans `ps`, `inspect` et les logs ; en variable, dans `inspect` et `exec env` ; en fichier monté ou `--secret`, nulle part sauf par le processus.
- `_FILE` ou `configtree` : l'environnement ne porte qu'un chemin.
- `podman secret` range sans chiffrer ; le vrai coffre est un gestionnaire de secrets avec rotation.
- Au build : `--mount=type=secret`, jamais `ARG` ; et le cache ne redemande pas un secret.

Vocabulaire

12-factor : douze règles pour des applications déployables, dont « config dans l'environnement ». — **configuration / secret** : ce qui varie par environnement / ce qui donne un accès. — **`ENV`** : variable par défaut inscrite dans l'image. — **`--env-file`** : fichier `CLE=valeur` chargé au `run`. — **relaxed binding** : traduction Spring Boot des propriétés en variables. — **profil Spring** : jeu de configuration activé par `SPRING_PROFILES_ACTIVE`. — **convention `_FILE`** : lire une valeur dans un fichier dont le chemin est en variable. — **`podman secret`** : secret géré par le moteur, monté dans

`/run/secrets/`. — **secret de build** : fichier disponible pendant une seule instruction `RUN`. — **gestionnaire de secrets** : service chiffré, journalisé, avec rotation.